PageRank: custom implementation, validation, and empirical analysis

From first principles to experimental results

Analysis of Algorithms Project

Yachay Tech

April 11, 2026

This presentation is based on the real outputs produced by our local Jupyter notebook.

Presentation roadmap

1. What problem PageRank solves
2. Basic concepts before the project
3. Meaning of the formulas
4. A small worked example
5. Our custom implementation
6. Validation against `NetworkX`
7. Damping, runtime, and sensitivity experiments
8. Conclusions and future work

Key takeaway

The goal of the project is not only to *use* PageRank, but to show that we understand its mathematics, implementation, and experimental behavior.

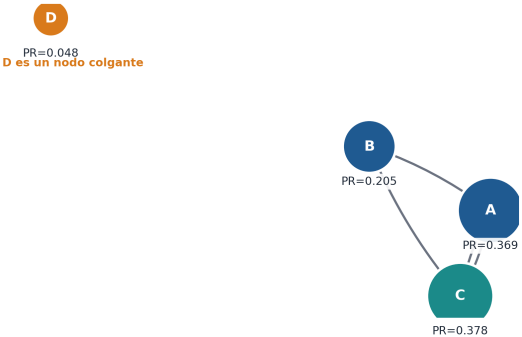
What question does PageRank answer?

- We want to rank nodes in a directed network.
- A node should be important if it is pointed to by important nodes.
- Not every link has the same value:
  - a link from a strong node transfers more authority;
  - if a node points to many neighbors, each outgoing vote is smaller.
- We must also handle dangling nodes, that is, nodes with no outgoing edges.

Intuition

PageRank can be seen as the long-term behavior of a random surfer moving through the network.

Grafo manual y ranking final de PageRank



Basic concepts before the project

Directed graph

A directed graph is a set of nodes connected by oriented edges:

$$A \to B$$

means that node A points to node B.

Degrees

For a node j:

$$\mathrm{outdeg}(j) = \text{number of outgoing edges from } j$$

$$\mathrm{indeg}(j) = \text{number of incoming edges to } j$$

Dangling node

A dangling node has:

$$\mathrm{outdeg}(j) = 0$$

This is a problem because the node does not know how to distribute its score.

Fix

If a node has no outgoing edges, we redistribute its probability uniformly across all nodes.

$$\begin{bmatrix} 1/N \\ \vdots \\ 1/N \end{bmatrix}$$

Transition matrix: what does the formula mean?

Definition

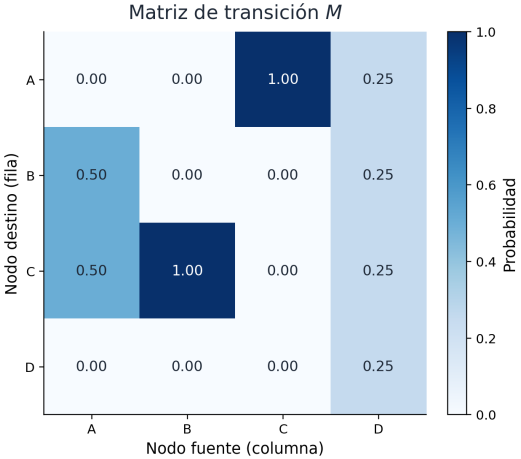
$$M_{ij} = \begin{cases} \dfrac{1}{\mathrm{outdeg}(j)} & \text{if } j \to i, \\ 0 & \text{otherwise.} \end{cases}$$

How to read it

- Column j: how node j distributes its score.
- Row i: how much score node i receives.
- Each column must sum to 1.

Meaning

The matrix M converts the graph into a probability transition system.



PageRank update: meaning of every symbol

Main iteration

$$\mathrm{r}^{(t+1)} = \frac{1-d}{N}\mathbf{1} + d\, M\mathrm{r}^{(t)}$$

Stopping rule

$$\left\| \mathrm{r}^{(t+1)} - \mathrm{r}^{(t)} \right\|_1 < \varepsilon$$

Symbols

- N: number of nodes
- M: transition matrix
- $\mathrm{r}^{(t)}$: score vector at iteration t
- d: damping factor
- $\mathbf{1}$: vector of ones
- ε: convergence tolerance
- $\|\cdot\|_1$: sum of absolute differences

Interpretation

The first term is **teleportation**. The second term follows the real links of the graph.

Why is teleportation necessary?

Without teleportation

Problems may appear:

- dangling nodes,
- closed cycles,
- unstable or trapped probability mass.

With teleportation

With probability d, the surfer follows a link.

With probability $1 - d$, the surfer jumps to any node uniformly.

Typical value

In practice, a standard value is:

$$d = 0.85$$

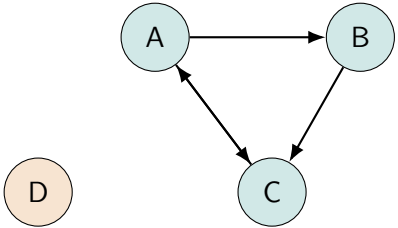
This balances:

- trust in the graph structure,
- protection against traps.

Consequence

Teleportation makes the process stable and helps guarantee a unique stationary distribution.

Small worked example: graph and setup



Observation

Node D is dangling because it has no outgoing edges.

Corrected transition matrix

$$M = \begin{bmatrix} 0 & 0 & 1 & 1/4 \\ 1/2 & 0 & 0 & 1/4 \\ 1/2 & 1 & 0 & 1/4 \\ 0 & 0 & 0 & 1/4 \end{bmatrix}$$

Initial vector

$$\mathrm{r}^{(0)} = \begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix} \qquad d = 0.85 \qquad \frac{1-d}{N} = 0.0375$$

Small worked example: first iteration

Apply the formula

$$\mathrm{r}^{(1)} = \frac{1-d}{N}\mathbf{1} + dM\mathrm{r}^{(0)}$$

$$\mathrm{r}^{(1)} = 0.0375 \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} + 0.85 \begin{bmatrix} 0.3125 \\ 0.1875 \\ 0.4375 \\ 0.0625 \end{bmatrix}$$

$$\mathrm{r}^{(1)} \approx \begin{bmatrix} 0.3031 \\ 0.1969 \\ 0.4094 \\ 0.0906 \end{bmatrix}$$

Interpretation

- C becomes the strongest node early.
- It receives score from both A and B.
- D stays low because it is dangling and has no incoming edges.

What happens next?

We repeat the same update until the vector changes very little.

Key takeaway

This small example shows exactly how the formula becomes numbers.

What we implemented from scratch

Implementation workflow

1. Build the directed graph.
2. Construct the transition matrix.
3. Fix dangling nodes.
4. Start from a uniform score vector.
5. Apply power iteration.
6. Stop when the L_1 error is below ε.
7. Compare with `NetworkX`.

Update rule

$$\mathbf{r}^{(t+1)} = \frac{1-d}{N}\mathbf{1} + dM\mathbf{r}^{(t)}$$

$$\left\| \mathbf{r}^{(t+1)} - \mathbf{r}^{(t)} \right\|_1 < \varepsilon$$

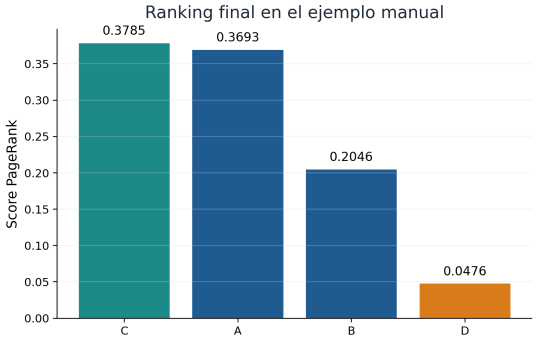
Interpretation

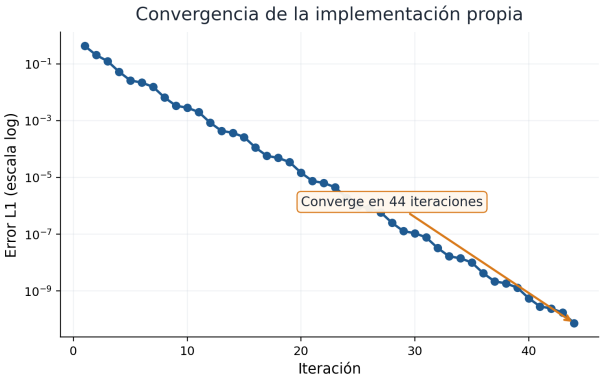
Each iteration redistributes authority through the graph and adds teleportation for stability.

Defense point

This was not a black-box experiment: we implemented the algorithm ourselves and then validated the output.

Manual example: final result and convergence

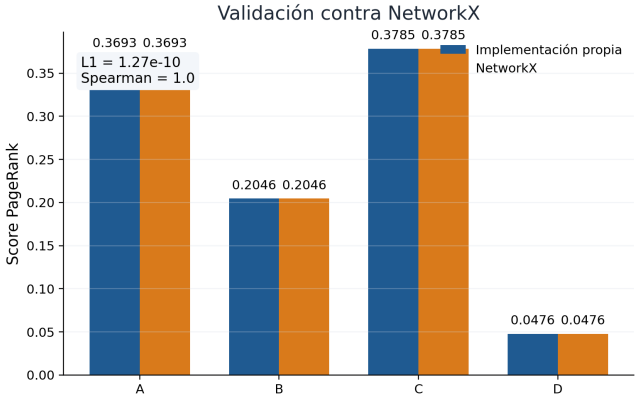




Reading the result

- C ranks first because it receives score from A and B.
- A ranks second because C transfers all of its authority to A.
- D ranks last because it is dangling and has no incoming support.

Validation against `NetworkX`



Validation metrics

L_1 distance	$\approx 1.27 \times 10^{-10}$
Max difference	$\approx 6.36 \times 10^{-11}$
Spearman corr.	1.0
Same ranking?	Yes

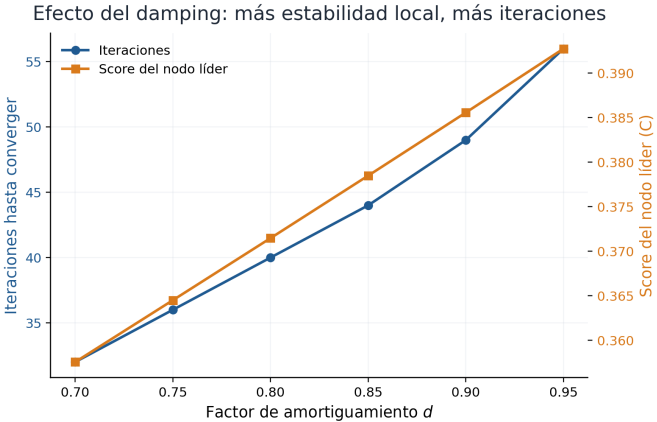
Final order

$$C > A > B > D$$

Defense message

Our output is numerically indistinguishable from the `NetworkX` output in the manual example.

Experiment 1: effect of the damping factor



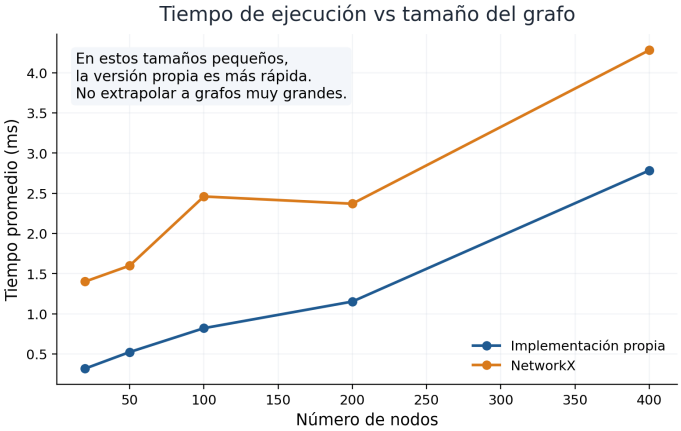
What we observed

- Larger d required more iterations.
- The top score increased slightly as d increased.
- The ranking order stayed unchanged in this example.

Correct interpretation

The smallest validation error near $d = 0.8$ is only a numerical detail. It is **not** evidence that $d = 0.8$ is scientifically superior.

Experiment 2: runtime vs graph size



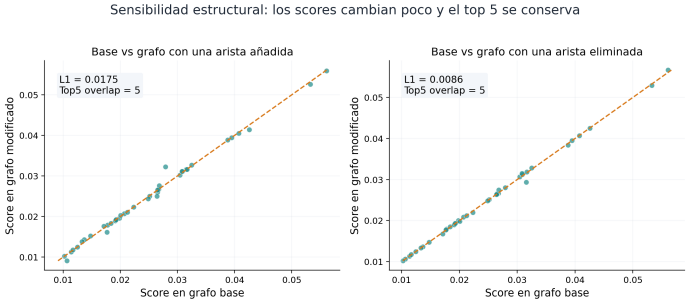
What the plot shows

- From 20 to 400 nodes, our implementation was faster than `NetworkX`.
- The mean number of iterations stayed around 24–27.
- Validation against `NetworkX` remained excellent.

What we should not overclaim

This does **not** prove that our implementation is better in every scenario. It only shows that, for these small dense experiments, the cost was low and competitive.

Experiment 3: sensitivity to structural changes



Results

- Add one edge:

$$L_1 = 0.0175$$

- Remove one edge:

$$L_1 = 0.0086$$

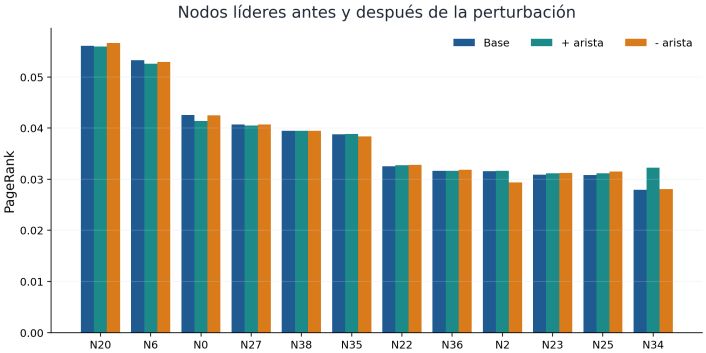
- In both cases:

$$\text{Top5 overlap} = 5$$

Interpretation

Scores do change, but not chaotically. The most important nodes remain the most important ones.

What happened to the top nodes?



Reading the figure

- The top nodes move only slightly.
- Local perturbations change values, but not the main hierarchy.
- This suggests reasonable ranking stability.

Key takeaway

PageRank is sensitive to structure, but the elite of the ranking was preserved in our experiment.

Main conclusions

What we achieved

1. We implemented PageRank from scratch with correct dangling-node handling.
2. We observed stable convergence through power iteration.
3. We validated the custom result against `NetworkX` with near-perfect agreement.
4. We showed how the damping factor changes convergence behavior.
5. We showed that structural perturbations affect scores, but the top nodes remained stable.

Final message

The project satisfies the three central goals of the course:

- **formal understanding**,
- **custom Python implementation**,
- **reproducible empirical analysis**.

Limitations and future work

Current limitations

- We used a **dense** matrix implementation.
- Runtime experiments only reached 400 nodes.
- We did not use a large real-world dataset yet.

Future work

- Sparse implementation for large graphs
- Evaluation on real networks
- Personalized PageRank and related variants

Thank you